

05

Cyber Security
Codec Technologies India
Internship 2026

Secure Web Application

Flask app mapped to OWASP
Top-10 (2021)

AT A GLANCE

OWASP Top-10

Controls

Argon2id

Password hash

JWT HS256

API auth

10 / 10

Recorded tests

5 / 60 s

Rate limit

5 present

Sec headers

no (argon2)

Re-run here

VERDICT

TESTS 10/10

RECORDED RUN

DEPENDENCY UNAVAILABLE

DEFENSIVE ONLY

DECLARED SUBSTITUTION

Eswar Mahalingam

Cyber Security Intern
Ghaziabad, UP, India
+91-9360548243
eswarmba05313@gmail.com
linkedin.com/in/eswar-mahalingam

Objective & brief mapping

I built a small but real Flask "Secure Notes" app to demonstrate defensive controls mapped one-to-one to the OWASP Top-10 (2021): Argon2id password hashing, server sessions, a JWT-protected API, role-based authorization, parameterised SQL, output escaping, CSRF protection, security headers, and login rate limiting. A per-user notes feature makes authorization meaningful (IDOR prevented).

Architecture

create_app() wires a before_request that loads the session user and an after_request that stamps five security headers. db.py holds parameterised SQLite; security.py holds Argon2 hashing, CSRF, the rate limiter, headers and RBAC; templates auto-escape via Jinja2.

```
run.py -> create_app()
  before_request: load session user
  after_request: CSP/HSTS/XFO/nosniff/Referrer
db.py (parameterised) | security.py (argon2, CSRF, RBAC, JWT)
```

Secure Notes

[Login](#) [Register](#)

Login

Username

Password

Log in

Educational secure-coding demo · OWASP Top 10 (2021)

The real login page served by the app (results/screenshot_login.png).

Reproduction & artefacts

Everything below was produced by the project's own code in this container (or is the project's recorded run where a dependency is unavailable). The commands that regenerate it:

```
python run.py # http://127.0.0.1:5000
python capture_screenshots.py # regenerate results/ screenshots
pytest -q # recorded: 10 passed (argon2 unavailable here)
```

The artefacts each figure and table above are drawn from:

- **test_report.txt** — 10/10 control tests
- **security_headers.txt** — live curl -I dump
- **screenshot_login.png / screenshot_logged_in.png** — real page renders

Method & implementation

Login clears the session first (fixation defence) and is rate-limited 5/60 s per IP+user. Note reads are scoped to owner_id, so guessing another user's id returns 404. POST /api/token issues an HS256 JWT (30-min expiry); GET /api/me needs a valid bearer token; the admin endpoint additionally requires the admin role.

Results

From results/test_report.txt: 10/10 controls proven. Each row below is a distinct test asserting one control end-to-end.

Control (OWASP)	Attack tried	Result
A03 Injection	alice' OR '1'='1 login	401
A03 XSS	stored <script>	escaped
A01 IDOR	user B reads user A note	404
A01 CSRF	POST without token	400
A07 Auth	tampered / missing JWT	401
A01 RBAC	user hits admin endpoint	403
A02 Storage	stored password prefix	\$argon2
A05 Rate limit	6th bad login	429

OWASP control -> implementation

OWASP	Control	Where
A01	owner-scoped reads, RBAC	views + security.py
A02	Argon2id hashing	security.py
A03	parameterised SQL, Jinja escape	db.py + templates
A05/A07	headers, rate limit, JWT	security.py

KEY TAKEAWAY A real Secure Notes app where each control is proven by a test: SQLi blocked, XSS escaped, IDOR 404, CSRF 400, JWT/RBAC enforced, Argon2 storage.

05

Cyber Security
Codec Technologies India
Internship 2026

Secure Web Application

Flask app mapped to OWASP
Top-10 (2021)

AT A GLANCE

OWASP Top-10

Controls

Argon2id

Password hash

JWT HS256

API auth

10 / 10

Recorded tests

5 / 60 s

Rate limit

5 present

Sec headers

no (argon2)

Re-run here

VERDICT

TESTS 10/10

RECORDED RUN

DEPENDENCY UNAVAILABLE

DEFENSIVE ONLY

DECLARED SUBSTITUTION

Eswar Mahalingam

Cyber Security Intern
Ghaziabad, UP, India
+91-9360548243
eswarmba05313@gmail.com
linkedin.com/in/eswar-mahalingam

Verification

Tests not re-run here — the argon2-cffi dependency is unavailable in this container and the app imports it at module load. The project's recorded run is **10 passed** (results/test_report.txt), which I did not modify. I verified the report artefact and the security_headers.txt dump are present and consistent.

Test group	Checks	Result (recorded)
Injection/XSS/IDOR	payloads neutralised	PASS
Auth / RBAC / CSRF	401/403/400 enforced	PASS
Storage / headers / limit	argon2, 5 headers, 429	PASS
Recorded total	10 tests	10/10

How I re-ran & verified in this container

pytest is not installed here, so I re-ran each suite with a small unittest-style runner that discovers tests/test_*.py, resolves fixtures (conftest, module-level @pytest.fixture, tmp_path, monkeypatch, request) and records pass / fail / skip. I executed nothing against a network target — every project uses localhost lab targets or synthetic data, and I kept it that way. Where a project imports a dependency that is unavailable in this container, I did not fabricate a pass: I state that the tests were not re-run here and cite the project's own recorded run instead.

Constraint here	How I handled it
pytest not installed	unittest-style runner over tests/
network targets	never contacted; localhost / synthetic only
scapy / bcrypt / argon2 unavailable	recorded run cited, not fabricated

Real-tool mapping, limitations & next steps

Each OWASP control is real; screenshots fell back to a documented renderer when Chromium was unreachable.

Ran here (recorded)	Real tool	Note
Flask test-client renders	Playwright + Chromium	download host blocked -> wkhtmltoimage
Argon2id hashing	argon2-cffi	unavailable in this container
in-memory rate limiter	Redis-backed limiter	single-process demo

Ethics, data integrity & declared substitutions

Defensive and educational. All data is created by the app at runtime; test users and notes are synthetic.

Declared substitution (copied from my README): Playwright's headless Chromium could not be installed here, so capture_screenshots.py renders the exact HTML the app serves through the Flask test client and rasterises it with wkhtmltoimage — the images are of the real pages. Dev-mode secret keys are labelled placeholders; set SECRET_KEY/JWT_SECRET from the environment before any deployment.

Learnings

Writing one test per OWASP control forced me to make each defence observable — a control you cannot demonstrate failing-safe is not really proven. The IDOR-returns-404 pattern (don't reveal that the record exists) and clearing the session on login were the two details I will carry into any future app.

Honest limitations & data-integrity notes

Honest limitations of this build, and how it maps to production tooling:

- The rate limiter is in-memory and per-process (not for a cluster).
- SQLite and the werkzeug dev server are used for simplicity, not production.
- Sessions are server-signed cookies; production might use server-side storage + JWT revocation.
- Dev-mode secret keys are labelled placeholders; set them from the environment.
- Serve over HTTPS so Secure cookies and HSTS take effect.
- Login clears the session first (fixation defence) and is rate-limited 5/60 s.
- Every response carries CSP, HSTS, X-Frame-Options, nosniff, Referrer-Policy.
- JWT is HS256 with a 30-minute expiry; admin endpoints add an RBAC check.
- Full test-suite runs in about 4 seconds (recorded).
- Stored password begins with \$argon2 (never plaintext).
- 6th bad login within the window returns 429.
- All five security headers present; X-Frame-Options: DENY.
- security_headers.txt is a live curl -I dump.
- JWT tampering and a missing token both return 401.
- A valid user hitting the admin endpoint gets 403 (RBAC).
- SQLi payload in login returns 401, not authenticated.
- Reference: OWASP Top 10 (2021) and the OWASP Cheat Sheets.